

252-0027

Einführung in die Programmierung Übungen

Woche 2: Git und EBNF

**Joran Bühler
Departement Informatik
ETH Zürich**

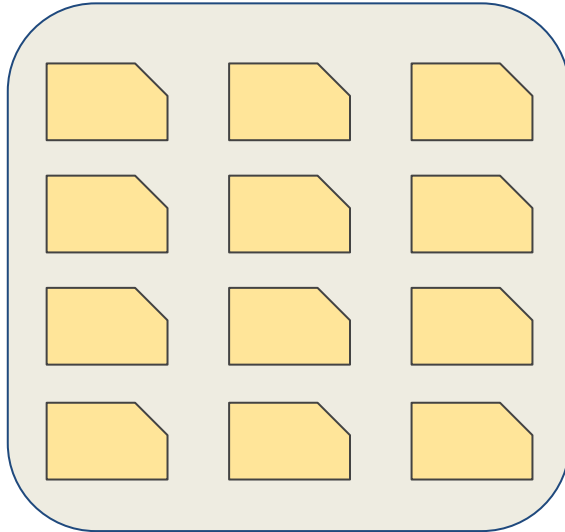
Ablauf

- Was ist GitHub?
- Übungsblatt 2
- EBNF Beispiel
- Ableitungen in EBNF
- Kahoot

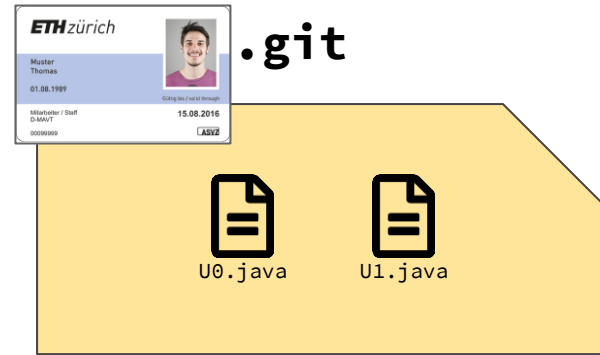
GitHub by ChatGPT

- **GitHub** ist eine Plattform, auf der man Programmcode speichern und mit anderen teilen kann.
Sie basiert auf **Git**, einem System, das alle Änderungen am Code speichert.
So kann man leicht zu alten Versionen zurückspringen, Fehler finden und im Team zusammenarbeiten.

Git Repository

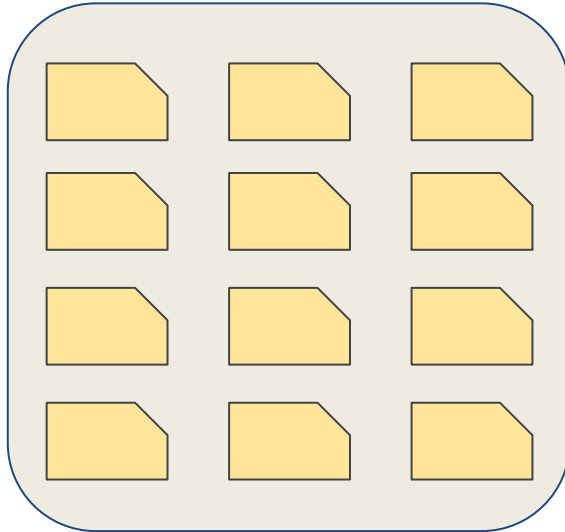


ETH Git-Server

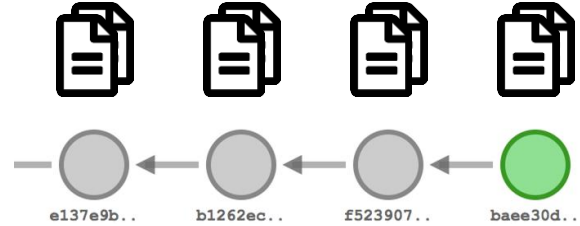


Jedes Repository auf dem Git-Server ist privat

Git Repository



ETH Git-Server



Ältester Commit

Neuester Commit

Jedes Repository auf dem Git-Server
Enthält eine Folge von **Commits** (die **History**)



.git

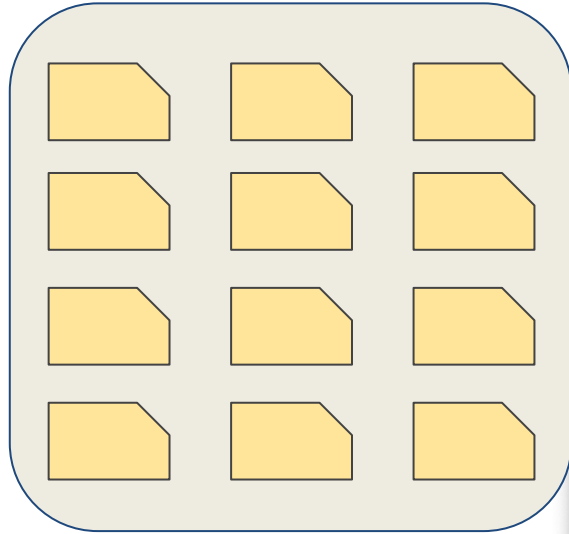


U0.java



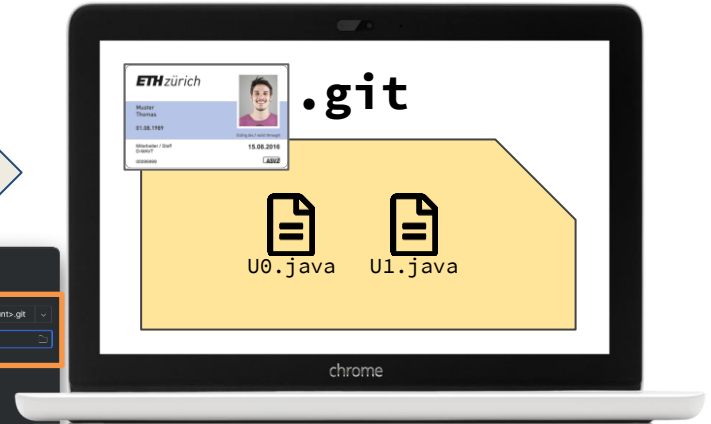
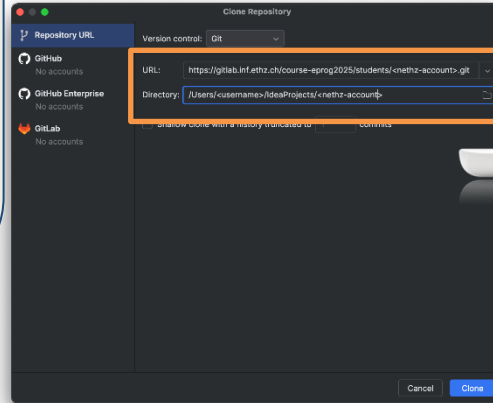
U1.java

Git Clone: Einmaliges Einrichten



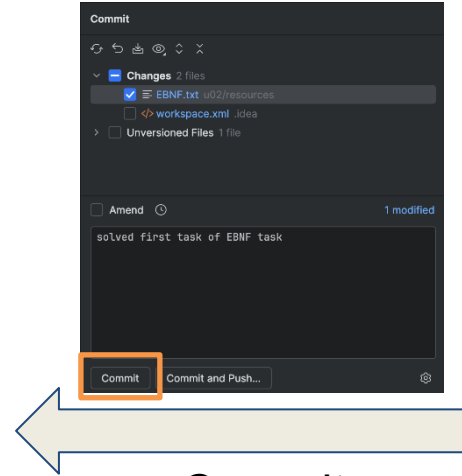
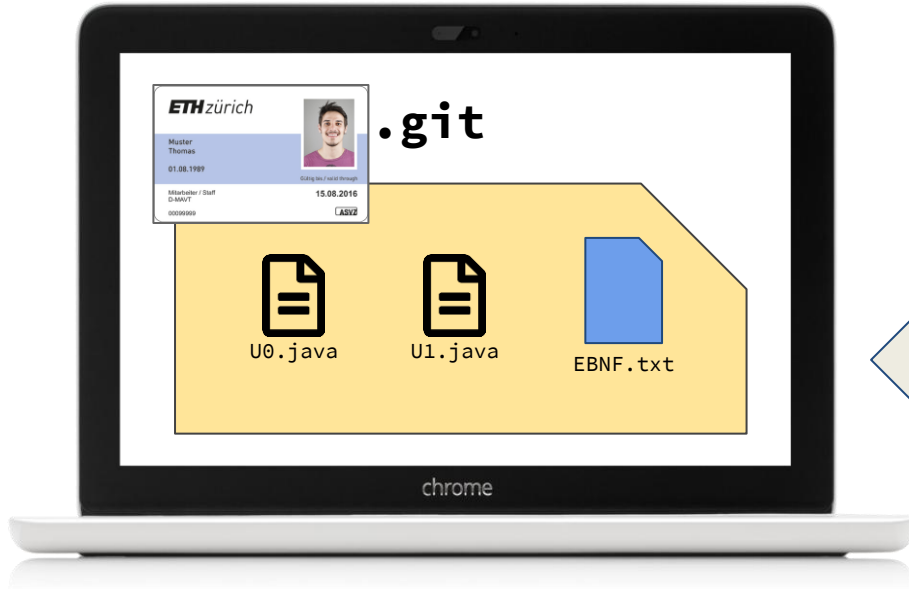
ETH Git-Server

Clone
Kopiert das ganze Repository
auf den eigenen Computer



Lokales Git-Repository

Git Commit: Fortschritt speichern

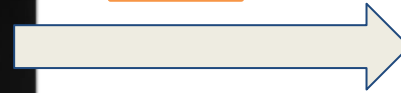
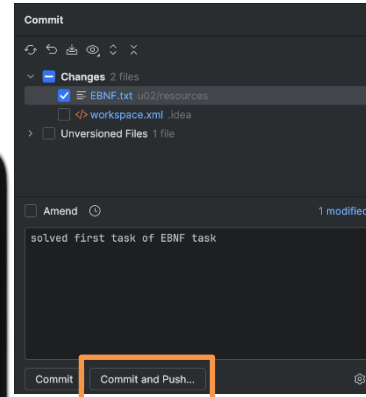
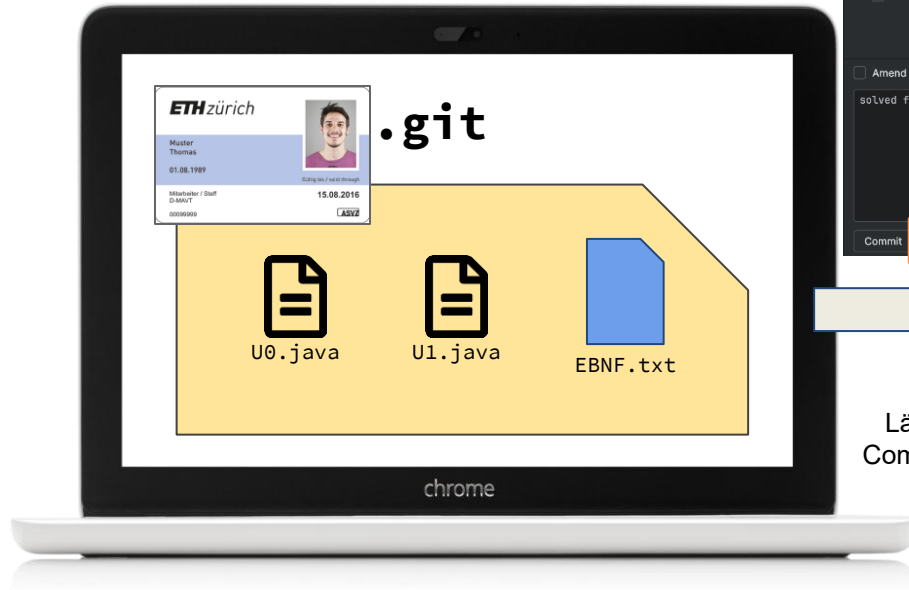


Commit
Fügt neuen Commit mit
Änderungen/neuen Dateien
der *lokalen* History hinzu

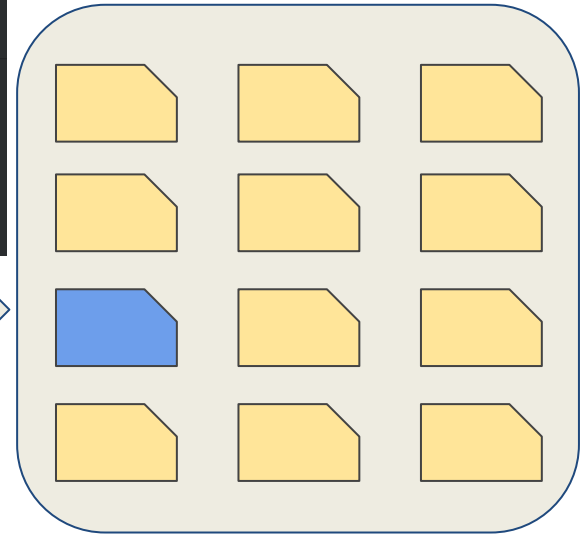


EBNF.txt

Git Push: Abgeben



Push
Lädt die neuen (lokalen)
Commits auf den Git-Server

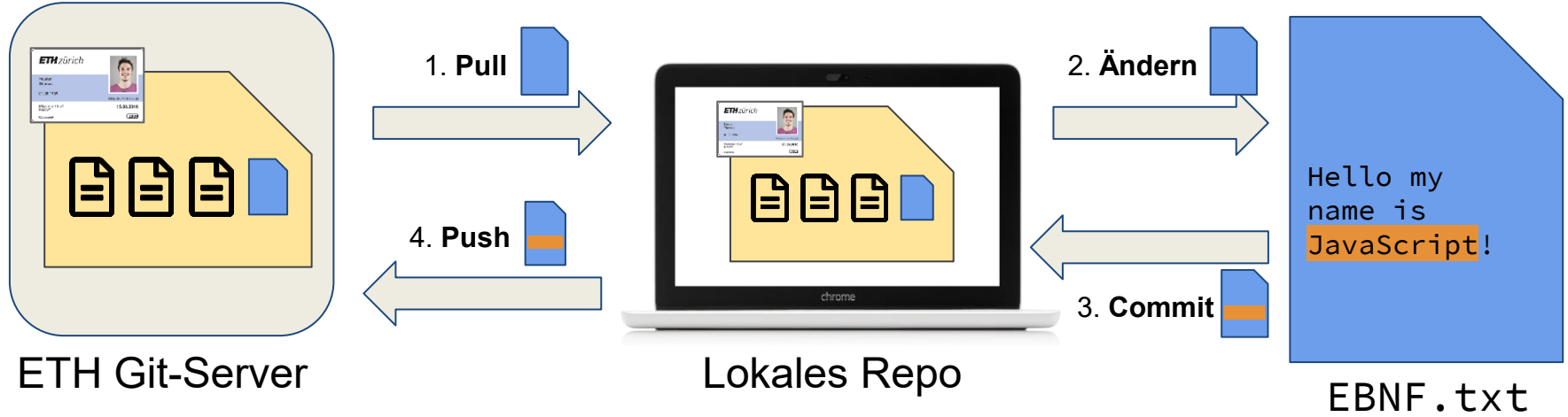


ETH Git-Server

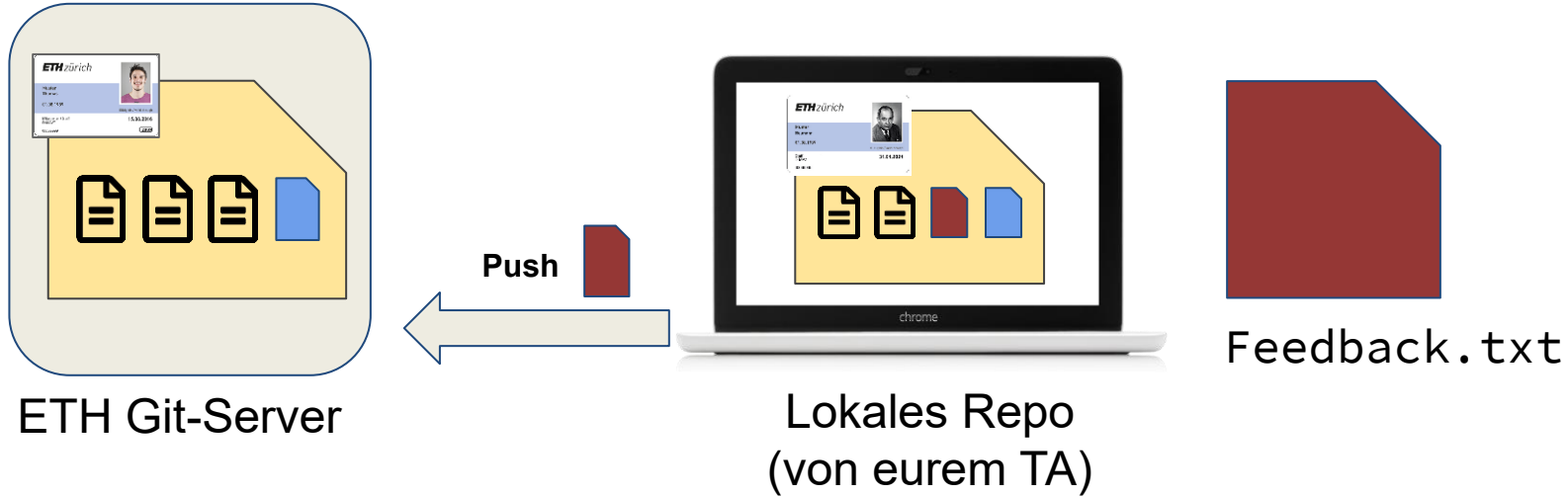
Git Pull: Aufgaben / Feedback laden



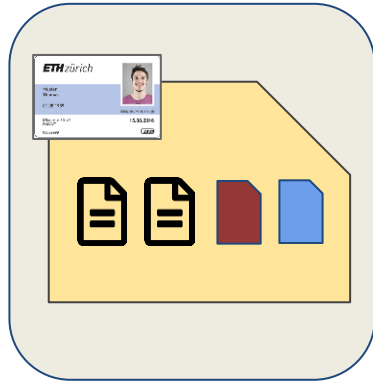
Git Pull/Push-Workflow



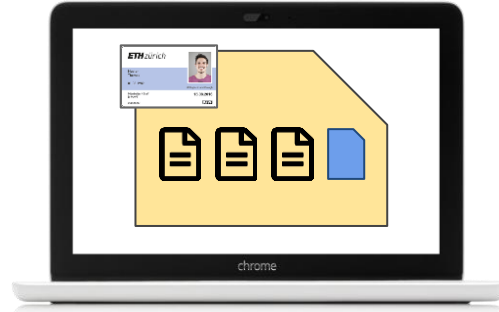
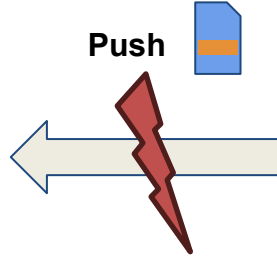
Git Repository ändert sich!



Git Merge Conflict



ETH Git-Server

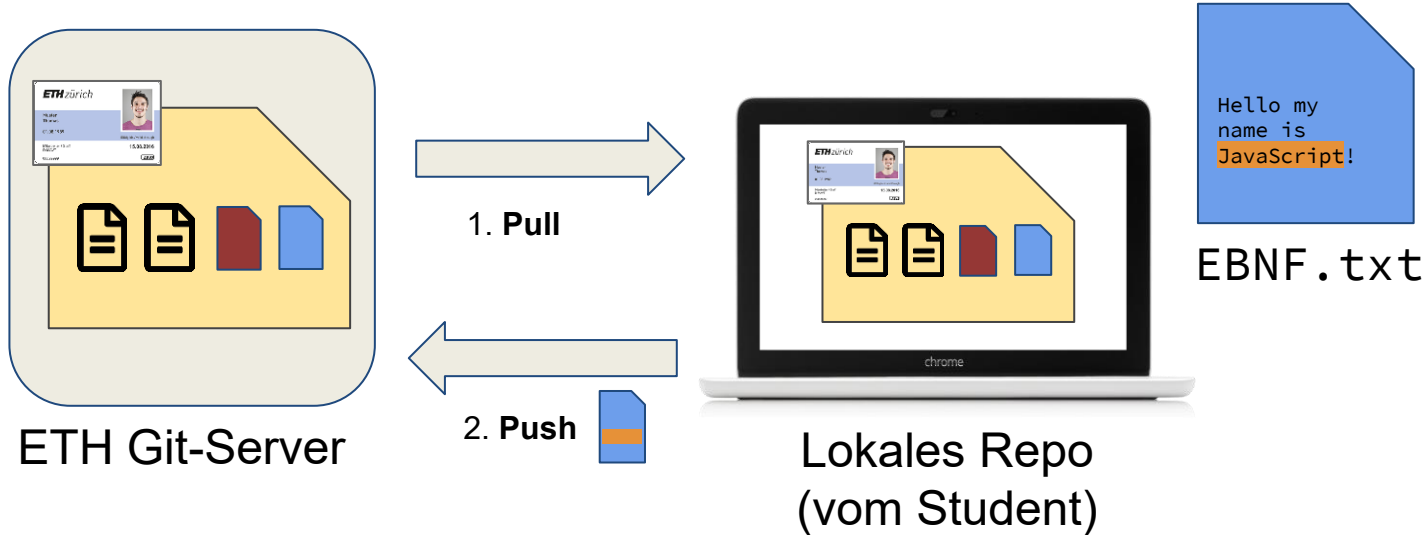


Lokales Repo
(vom Student)



EBNF.txt

Git Merge Conflict verhindern



Immer zuerst pull dann push!

Häufige Git-Fehler vermeiden

- Vor erster Benutzung auf gitlab.inf.ethz.ch anmelden
- In U02 Aufgabe 4, <nethz-account> im Link inklusive <> ersetzen
- In U02 Aufgabe 4, bei Problemen Link per Hand abtippen
- Files/Directories/Projects/... immer in IntelliJ löschen bzw. ändern (nie im File Explorer/Finder)
- Immer Pull und dann Push (checke Online ob Änderungen sichtbar sind)
- “Commit und Push” statt nur “Commit”
 - Falls Sie auf Commit geklickt haben, drücken Sie manuell per Rechtsklick auf das Repository und wählen Sie Git -> Push aus. Importieren bei neuen Projects nicht vergessen!

Git: Clone, Aus- und Einchecken

(Demo)

EBNF – Beispiel Programmiersprache

```
<programm> <= PROGRAM <bezeichner>
      BEGIN
      { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := ( (<zahl>|<bezeichner>) |<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9
```

Wie prüfe ich, was legal ist?

1. Informeller Beweis
2. Tabellen
3. Ableitungsbaum
4. Graphische Darstellung

JETZT


```
<programm> <= PROGRAM <bezeichner>
      BEGIN
        { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := (<zahl>|<bezeichner>|<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9
```

PROGRAM DEMO1

BEGIN

A0:=3;

B:=+45;

C:=A;

D123:=B34A;

TEXTZEILE:="HALLO";

END

```

<programm> <= PROGRAM <bezeichner>
      BEGIN
        { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := (<zahl>|<bezeichner>|<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9

```

PROGRAM DEMO1
BEGIN

```

A0:=3;
B:=+45;
H:=-100023;
C:=A;
D123:=B34A;
ESEL:=GIRAFFE;
TEXTZEILE:="HALLO";

```

END



```

<programm> <= PROGRAM <bezeichner>
      BEGIN
        { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := (<zahl>|<bezeichner>|<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9

```

PROGRAM DEMO2

BEGIN

myVariable3 := 5

2GOOD2BETRUE := 6ER

END

```
<programm> <= PROGRAM <bezeichner>
      BEGIN
        { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := (<zahl>|<bezeichner>|<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9
```

PROGRAM DEMO2

BEGIN

myVariable3 := 5

2GOOD2BETRUE := 6ER

END



```
<programm> <= PROGRAM <bezeichner>
      BEGIN
        { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := (<zahl>|<bezeichner>|<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9
```

PROGRAM DEMO3

BEGIN

GOODNAME := +-5

MARK := 5.5

END

```
<programm> <= PROGRAM <bezeichner>
      BEGIN
        { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := (<zahl>|<bezeichner>|<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9
```

PROGRAM DEMO3

BEGIN

GOODNAME := +-5

MARK := 5.5

END



Repetition: Ableitungen

- Ableitungstabelle
 - Erste Zeile ist Startregel
 - Letzte Zeile ist Zeichenfolge
 - Übergang zwischen zwei Zeilen entspricht Ableitungsschritt
- Ableitungsbaum
 - Wurzel ist Namen der Startregel
 - Blätter sind Zeichen
 - Verbindungen stehen für einen Ableitungsschritt

Beispiel: Ableitung von -127 als Tabelle

(R1) <digit> ← 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

(R2) <integer> ← [+ | -] <digit> {<digit>}

Beispiel: Ableitung von -127 als Tabelle

(R1) <digit> ← 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

(R2) <integer> ← [+ | -] <digit> {<digit>}

<integer> ← [+ | -] <digit> {<digit>}

 ← + | - <digit> {<digit>}

 ← - <digit> {<digit>}

 ← - <digit> <digit> <digit>

 ← - 1 <digit> <digit>

 ← - 1 2 <digit>

 ← - 1 2 7

(R2)

Option gewählt

- gewählt

2 mal wiederholt

(R1) und 1 gewählt

(R1) und 2 gewählt

(R1) und 7 gewählt

Beispiel: Ableitung von -127 als Baum

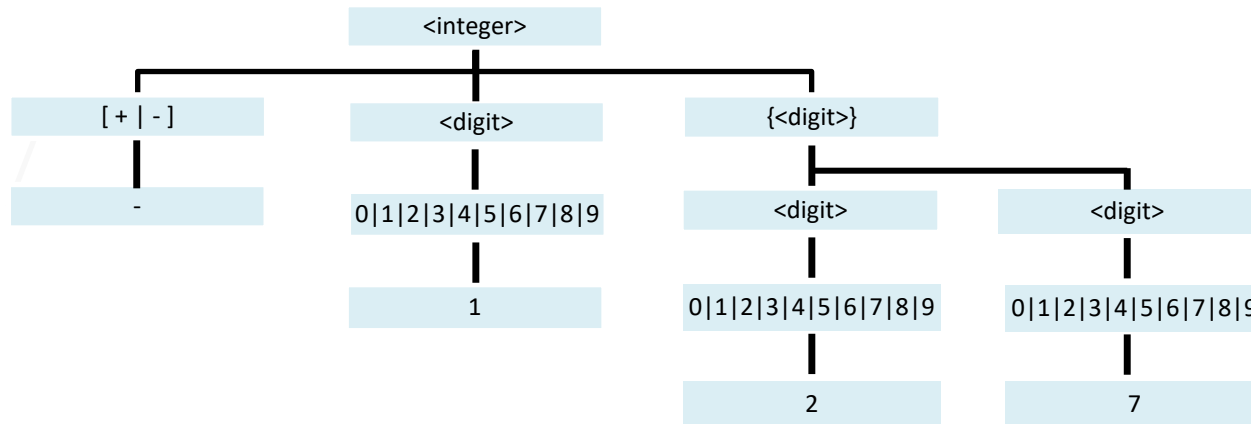
<digit> ← 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<integer> ← [+ | -] <digit> {<digit>}

Beispiel: Ableitung von -127 als Baum

$\langle \text{digit} \rangle \leftarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$\langle \text{integer} \rangle \leftarrow [+ \mid -] \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \}$



Kahoot - Prüfungsaufgabe

- **FS 2020 Aufgabe 5**
- **8/60 Punkte**
 - Damals 60 min Zeit -> 8 min für die Aufgabe